

SmartCampus

Resource & Energy Analytics System

Complete Project Master Guide

Everything you need to confidently present this project and answer any question — concepts, tables, all code files with snippets

Roll No	Name
24BDS009	Rakhal Krishna
24BDS010	Vineel Tej
24BDS030	Rishikesh
24BDS039	Arjun

IIIT Dharwad · CSE Department · DA264 Database Management · 2025–2026

1. What Is This Project? — The Big Picture

SmartCampus is a full-stack database project simulating an intelligent campus management system for IIIT Dharwad. It tracks **energy consumption**, **room bookings**, **equipment**, **maintenance issues**, and **access control** across all campus buildings, visualised in a live React dashboard. It covers all four units of DA264.

1.1 Technology Stack

Layer	Technology	Purpose
OLTP Database	SQL Server 2022 (SSMS)	12 operational tables, stored procedures, triggers, indexes, views
Data Warehouse	SQL Server (same instance)	Star+Snowflake schema — OLAP fact & dimension tables
ETL Pipeline	Python pandas + SQLAlchemy	Automated incremental load OLTP → Warehouse with IQR cleaning
Backend API	Python FastAPI + uvicorn	JWT-authenticated REST API on port 8000
Frontend	React + Vite + Tailwind CSS	9-page live dashboard with charts, tables, data-entry forms
Auth	JWT HS256 via python-jose	Token-based login; admin vs viewer role-based access

1.2 Data Flow

Browser → React calls GET /api/dashboard-stats (JWT) → FastAPI queries SQL Server → returns JSON → charts render. When user submits a Live Operations form → React calls POST endpoint → FastAPI executes SQL INSERT → DB triggers fire → dashboard auto-refreshes every 28 seconds.

2. Core Database Concepts Used in This Project

2.1 Relational Database Basics

A relational database stores data in **tables** (rows + columns). Tables are linked via **foreign keys**. SQL Server is our RDBMS — all 12 operational tables live in **SmartCampusDB**.

Primary Key (PK): Uniquely identifies every row; cannot be NULL. Example: room_id=109 always means "L201 Computer Lab".

Foreign Key (FK): References a PK in another table. Enforces referential integrity — you cannot insert a meter pointing to a nonexistent room. SQL Server raises a FK violation error and rejects the insert.

2.2 Normalisation — 1NF to BCNF

Normal Form	Rule	How We Applied It
1NF	Atomic values; no repeating groups	EQUIPMENT_TYPE is a separate table — not a comma list inside EQUIPMENT
2NF	No partial dependencies on composite key	All attributes depend on the full PK only
3NF	No transitive dependencies	Building info lives in BUILDING table, not repeated in ROOM — chain: ROOM.floor_id → FLOOR.building_id → BUILDING
BCNF	Every determinant is a candidate key	All 12 tables satisfy this; every non-key column is determined solely by PK

✓ **Tip:** If asked "Why does this matter?": Storing dept_name in ROOM directly means updating a department name requires changing hundreds of rows — an update anomaly. Normalisation prevents this.

2.3 ACID Properties

Property	Meaning	Our Project Example
Atomicity	All steps succeed or all fail (rollback)	Booking + Access Log inserted together; FK failure on Access Log rolls back the Booking too
Consistency	DB transitions between valid states only	CHECK constraint: end_time > start_time prevents invalid booking times
Isolation	Concurrent transactions do not see each other's uncommitted data	READ COMMITTED (default); SERIALIZABLE for pending-count queries
Durability	Committed data survives crashes	SQL Server writes to transaction log; data persists after restart

2.4 OLTP vs OLAP

OLTP = Online Transaction Processing. Live database. Handles real-time inserts/updates. Write-optimised. Our 12-table SmartCampusDB is OLTP.

OLAP = Online Analytical Processing. Data Warehouse. Stores historical aggregated data. Read-optimised for GROUP BY and aggregation. Our Star Schema (fact_energy_consumption + dimension tables) is OLAP. Data moves OLTP → OLAP via ETL pipeline.

2.5 Stored Procedures vs Triggers

Stored Procedure: Named, pre-compiled SQL program called explicitly (EXEC usp_AddBooking ...). We have usp_AddBooking (conflict check + insert) and usp_UpdateBookingStatus (validate + update).

Trigger: Auto-fires on DML events (INSERT/UPDATE/DELETE) on a specific table — no explicit call. We have 3: trg_booking_insert (audit on new booking), trg_booking_update (audit on status change), trg_booking_nodelete (block deletion of Approved bookings).

3. Every Table Explained — What It Is and Why We Built It

3.1 DEPARTMENT

Stores the 3 academic departments (CSE, ECE, DSAI). **Why separate?** Buildings and users reference dept_id FK instead of repeating the name, preventing update anomalies. **Circular FK:** head_fac_id points to FACULTY — each department has a head. This created a circular dependency solved by creating DEPARTMENT with NULL head_fac_id first, then FACULTY, then adding the FK via ALTER TABLE.

3.2 USERS, STUDENT, FACULTY — IS-A Hierarchy

USERS is the supertype with common attributes (name, email, phone, user_type). **STUDENT** extends USERS adding roll_no, year_of_study. **FACULTY** extends USERS adding designation, employee_no. Both subtypes use user_id as both PK and FK simultaneously — shared identity with USERS.

✓ **Tip:** "Why not one table?" — Faculty has no roll_no; students have no employee_no. One table means 50% NULL columns — a design violation. IS-A separation is cleaner.

3.3 BUILDING → FLOOR → ROOM (Spatial Hierarchy)

5-level chain: DEPARTMENT → BUILDING (1:N) → FLOOR (1:N) → ROOM (1:N) → ENERGY_METER (1:N) → ENERGY_READING. Each level is a separate table with FK to the level above. This prevents transitive dependencies — ROOM cannot transitively depend on building_name if FLOOR sits between them.

3.4 EQUIPMENT_TYPE and EQUIPMENT

EQUIPMENT_TYPE: Lookup table for categories (Desktop, GPU Server, Projector, AC...) with power_kw. **Why separate?** Deletion anomaly prevention — deleting all Desktop PCs from EQUIPMENT would not erase the knowledge that "Desktop Computer" is a valid category if it lives separately.

EQUIPMENT: Individual physical items with serial_no, status (Active/Inactive/Under Repair/Decommissioned, enforced by CHECK), room_id FK, and type_id FK.

3.5 ENERGY_METER and ENERGY_READING

ENERGY_METER: Physical sensor in a room. meter_type can be Electricity/Solar/Water/Gas (CHECK constraint). is_active BIT allows decommissioning without data deletion.

ENERGY_READING: Highest-volume table (~500+ rows). IDENTITY(1,1) PK — auto-assigned. Stores kwh_consumed, voltage, peak_flag (1=peak hours 8-18, 0=off-peak), timestamp, and meter_id FK. Feeds OLTP dashboard and ETL pipeline.

3.6 TIME_SLOT

Lookup table for named scheduling windows (e.g., "Morning Slot 1" = Monday 08:00–09:00). BOOKING.slot_id FK links to predefined slots. slot_id is nullable in BOOKING — ad-hoc bookings without a predefined slot are allowed.

3.7 BOOKING (M:N Junction Table)

Resolves the Many-to-Many between USERS and ROOM. One user books many rooms; one room is booked by many users. Holds start_time, end_time, purpose, status (Pending/Approved/Rejected/Cancelled/Completed). IDENTITY(1,1) PK — backend never specifies booking_id. CHECK constraint: end_time > start_time. AUDIT_TRAIL is populated automatically by triggers on every INSERT/UPDATE.

✓ **Tip:** "How do you prevent double-booking?" — usp_AddBooking checks: SELECT COUNT(*) FROM BOOKING WHERE room_id=? AND status NOT IN (Cancelled,Rejected) AND start_time < @end AND end_time > @start. If count > 0 → RAISERROR + ROLLBACK.

3.8 MAINTENANCE_TICKET (Weak Entity)

Records physical problems in rooms — e.g., "GPU Server overheating in L301". **Why weak entity:** ticket is meaningless without its room — room_id is NOT NULL and is part of logical identity. priority (Critical/High/Medium/Low) and status (Open/In Progress/Resolved/Closed) enforced by CHECK constraints. MTTR is computed as AVG(DATEDIFF(hour, reported_date, resolved_at)) per priority tier.

3.9 ACCESS_LOG (M:N Junction)

Logs every physical room entry: who, which room, when (entry_time), how (RFID/PIN/Biometric/Manual), how long (duration_min). auth_method validated by CHECK constraint. After-hours entries (past 10 PM) flagged on Security dashboard.

3.10 AUDIT_TRAIL (System Log)

Never populated manually — entirely fed by triggers. audit_id is UNIQUEIDENTIFIER (NEWID()) — globally unique even across distributed systems. Captures action_type (INSERT/UPDATE), old_value → new_value, timestamp, booking_id. Provides complete change history for accountability.

4. DBMS.sql — Complete Code Walkthrough

DBMS.sql is the master SQL script with 12 sections. Run it once in SSMS to create the entire database from scratch: tables, seed data, views, indexes, advanced queries, transactions, stored procedures, triggers, data warehouse, ETL, and OLAP queries.

4.1 Database Creation + Circular FK Solution

```
CREATE DATABASE SmartCampusDB;
GO
USE SmartCampusDB;
GO

-- Step 1: DEPARTMENT with NULL head (circular dep can't resolve yet)
CREATE TABLE DEPARTMENT (
    dept_id      INT PRIMARY KEY,
    dept_name    VARCHAR(100) NOT NULL,
    head_fac_id  INT NULL      -- FK added after FACULTY exists
);

-- Step 2: FACULTY (DEPARTMENT already exists, so dept_id FK works)
CREATE TABLE FACULTY (
    faculty_id   INT PRIMARY KEY,
    dept_id      INT NOT NULL,
    CONSTRAINT fk_fac_dept FOREIGN KEY (dept_id) REFERENCES DEPARTMENT(dept_id)
);

-- Step 3: NOW add the circular FK safely
ALTER TABLE DEPARTMENT
    ADD CONSTRAINT fk_dept_head
    FOREIGN KEY (head_fac_id) REFERENCES FACULTY(faculty_id);
```

4.2 IDENTITY Columns

```
CREATE TABLE ENERGY_READING (
    reading_id   INT IDENTITY(1,1) PRIMARY KEY, -- auto-increments: 1,2,3...
    timestamp    DATETIME NOT NULL,
    kwh_consumed FLOAT NOT NULL,
    peak_flag    BIT NOT NULL DEFAULT 0,
    meter_id     INT NOT NULL,
    CONSTRAINT chk_kwh CHECK (kwh_consumed >= 0),
    CONSTRAINT chk_voltage CHECK (voltage IS NULL OR voltage > 0),
    CONSTRAINT fk_rdg_meter FOREIGN KEY (meter_id) REFERENCES ENERGY_METER(meter_id)
);
```

BOOKING, MAINTENANCE_TICKET, ACCESS_LOG all use IDENTITY(1,1) too. The backend INSERT never specifies the PK — SQL Server assigns it automatically. This prevents PK conflicts in multi-user environments.

4.3 Views

```
-- Virtual table: aggregated energy per room
CREATE VIEW vw_ROOM_ENERGY_USAGE AS
SELECT R.room_no, R.room_type,
    COUNT(E.reading_id) AS total_readings,
    SUM(E.kwh_consumed) AS total_kwh,
    AVG(E.kwh_consumed) AS avg_kwh_per_hour,
    MAX(E.kwh_consumed) AS peak_kwh
FROM ENERGY_READING E
JOIN ENERGY_METER M ON E.meter_id = M.meter_id
JOIN ROOM R ON M.room_id = R.room_id
GROUP BY R.room_id, R.room_no, R.room_type;

-- Backend API queries this view directly:
-- SELECT * FROM vw_ROOM_ENERGY_USAGE ORDER BY total_kwh DESC
```

4.4 Indexes

```
-- Speed up timestamp-filtered energy queries (most common)
CREATE INDEX idx_energy_timestamp ON ENERGY_READING(timestamp);

-- Speed up booking conflict check (room + time range scan)
CREATE INDEX idx_booking_conflict ON BOOKING(room_id, start_time, end_time);

-- Partial index: only active meters (smaller, faster lookup)
CREATE INDEX idx_active_meters ON ENERGY_METER(room_id) WHERE is_active = 1;

-- Maintenance dashboard filter
CREATE INDEX idx_ticket_status ON MAINTENANCE_TICKET(status, priority);
```

✓ **Tip:** "Why not index everything?" Indexes slow down INSERT/UPDATE (must update index too) and use disk space. Only index columns frequently in WHERE, JOIN, or ORDER BY.

4.5 LAG Window Function — Anomaly Detection

```
SELECT meter_id, timestamp, kwh_consumed,
       LAG(kwh_consumed) OVER (PARTITION BY meter_id ORDER BY timestamp) AS prev_kwh,
       kwh_consumed
       - LAG(kwh_consumed) OVER (PARTITION BY meter_id ORDER BY timestamp) AS delta_kwh,
CASE
    WHEN kwh_consumed
        - LAG(kwh_consumed) OVER (PARTITION BY meter_id ORDER BY timestamp) > 2
    THEN 'SPIKE DETECTED'
    ELSE 'Normal'
END AS anomaly_flag
FROM ENERGY_READING;
-- LAG(col) looks at the PREVIOUS row's value (same meter, ordered by time)
-- PARTITION BY meter_id means LAG resets for each meter
```

4.6 CTE — Building Hierarchy Drill-Down

```
WITH BuildingHierarchy AS (
    SELECT B.bld_name, F.floor_label, R.room_no, R.capacity, R.room_id
    FROM BUILDING B
    JOIN FLOOR F ON F.building_id = B.building_id
    JOIN ROOM R ON R.floor_id = F.floor_id
)
SELECT bld_name, floor_label, room_no, capacity,
       (SELECT COUNT(*) FROM EQUIPMENT WHERE room_id = BH.room_id) AS equipment_count,
       (SELECT COUNT(*) FROM ENERGY_METER WHERE room_id = BH.room_id) AS meters_installed
FROM BuildingHierarchy BH
ORDER BY bld_name, floor_label;
```

4.7 Stored Procedure — usp_AddBooking

```
CREATE OR ALTER PROCEDURE usp_AddBooking
    @start_time DATETIME, @end_time DATETIME,
    @purpose VARCHAR(200), @room_id INT, @user_id INT
AS BEGIN
    SET NOCOUNT ON;
    IF @end_time <= @start_time
        RAISERROR('End time must be after start time.', 16, 1);

    BEGIN TRANSACTION;
    BEGIN TRY
        -- Conflict check: overlapping approved/pending booking for same room?
        IF EXISTS (
            SELECT 1 FROM BOOKING
            WHERE room_id = @room_id AND status IN ('Pending', 'Approved')
              AND start_time < @end_time AND end_time > @start_time
        )
        BEGIN
            -- Conflict exists, do nothing
        END
    END TRY
    BEGIN CATCH
        -- Conflict exists, do nothing
    END CATCH
    COMMIT;
```

```

        RAISERROR('Room already booked for this slot.', 16, 1);
        ROLLBACK TRANSACTION; RETURN;
    END
    -- Safe to insert
    INSERT INTO BOOKING (start_time, end_time, purpose, status, room_id, user_id)
    VALUES (@start_time, @end_time, @purpose, 'Pending', @room_id, @user_id);
    COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
    THROW; -- re-raise the error to the caller
END CATCH
END;

```

4.8 Triggers

```

-- AFTER INSERT trigger: auto-write to AUDIT_TRAIL
CREATE OR ALTER TRIGGER trg_booking_insert ON BOOKING AFTER INSERT
AS BEGIN
    SET NOCOUNT ON;
    -- 'inserted' is SQL Server's virtual table holding the new row(s)
    INSERT INTO AUDIT_TRAIL (audit_id, action_type, timestamp, new_value, booking_id)
    SELECT NEWID(), 'INSERT', GETDATE(),
           'New booking - Status: ' + i.status,
           i.booking_id
    FROM inserted i;
END;

-- INSTEAD OF DELETE trigger: block deletion of Approved bookings
CREATE OR ALTER TRIGGER trg_booking_nodelete ON BOOKING INSTEAD OF DELETE
AS BEGIN
    IF EXISTS (SELECT 1 FROM deleted WHERE status = 'Approved')
    BEGIN
        RAISERROR('Cannot delete Approved booking. Cancel it first.', 16, 1);
        ROLLBACK; RETURN;
    END
    -- If no Approved rows in the delete set, allow it
    DELETE FROM BOOKING WHERE booking_id IN (SELECT booking_id FROM deleted);
END;

```

✓ **Tip:** "inserted" and "deleted" are SQL Server magic virtual tables inside triggers. "inserted" = new rows (for INSERT/UPDATE). "deleted" = old rows (for UPDATE/DELETE).

5. seed_bulk.sql — Bulk Data Generation

seed_bulk.sql fills the database with hundreds of realistic rows to make dashboard charts look meaningful. It uses SQL generation tricks to avoid writing hundreds of INSERTs manually.

5.1 What It Inserts

Table	Count	Why
USERS	109 (76 Students, 15 Faculty, 15 Staff, 3 Admin)	Fills User Analytics charts
BOOKING	200 across 6 months	Monthly Trend, Pivot Table, Registry
ENERGY_READING	500+ readings across all meters	Energy heatmap, anomaly detection, Roll-Up
MAINTENANCE_TICKET	10 tickets of varying priority	Maintenance queue, SLA chart, aging
ACCESS_LOG	208 entries	Access & Security charts
EQUIPMENT	110 items	Equipment analytics

5.2 Realistic Data Generation Technique

```
-- Generate energy readings: every meter, every hour for 30 days
-- CHECKSUM(NEWID()) generates a pseudo-random integer
INSERT INTO ENERGY_READING (meter_id, timestamp, kwh_consumed, voltage, peak_flag)
SELECT
    m.meter_id,
    DATEADD(HOUR, h.n, '2025-01-01'),      -- timestamp for each hour
    CASE WHEN h.n % 24 BETWEEN 8 AND 18
        THEN 4.0 + (ABS(CHECKSUM(NEWID())) % 30) / 10.0 -- 4.0-7.0 kwh peak
        ELSE 1.0 + (ABS(CHECKSUM(NEWID())) % 15) / 10.0 -- 1.0-2.5 kwh off-peak
    END,
    218 + (ABS(CHECKSUM(NEWID())) % 15), -- voltage 218-232V
    CASE WHEN h.n % 24 BETWEEN 8 AND 18 THEN 1 ELSE 0 END
FROM ENERGY_METER m
CROSS JOIN (
    SELECT TOP 720 ROW_NUMBER() OVER (ORDER BY (SELECT NULL))-1 AS n
    FROM sys.objects
) h;
-- 7 meters × 720 hours = 5,040 rows with one statement
```

CROSS JOIN between meters and hours creates one row per combination — 7 meters × 720 hours = 5,040 readings. CHECKSUM(NEWID()) gives a different random value each row.

■ **Note:** Run seed_bulk.sql AFTER DBMS.sql — tables must already exist.

6. transaction.sql — ACID and Concurrency Demos

transaction.sql demonstrates all Unit III concepts with runnable SQL in SSMS, using SmartCampus tables as the real-world context.

6.1 What is a Transaction?

A sequence of SQL statements treated as one unit. Starts with BEGIN TRANSACTION, ends with COMMIT (success) or ROLLBACK (undo). Guarantee: ALL succeed, or NONE take effect.

6.2 Atomicity Demo

```
-- Test 1: Both steps succeed → COMMIT
BEGIN TRANSACTION;
  BEGIN TRY
    INSERT INTO BOOKING (start_time,end_time,purpose,status,room_id,user_id)
    VALUES ('2025-06-01 09:00','2025-06-01 10:00','Demo','Pending',104,1);
    INSERT INTO ACCESS_LOG (entry_time,access_method,room_id,user_id)
    VALUES ('2025-06-01 08:58','RFID',104,1);
    COMMIT TRANSACTION;
    PRINT 'Both inserts committed.';
  END TRY BEGIN CATCH
    ROLLBACK TRANSACTION;
    PRINT 'ROLLED BACK: ' + ERROR_MESSAGE();
  END CATCH;

-- Test 2: Second step fails (room_id 9999 doesn't exist → FK violation)
-- The booking from Step 1 is ALSO rolled back – Atomicity confirmed
BEGIN TRANSACTION;
  BEGIN TRY
    INSERT INTO BOOKING (... , room_id) VALUES (... , 104);    -- succeeds
    INSERT INTO ACCESS_LOG (... , room_id) VALUES (... , 9999); -- FAILS (FK)
    COMMIT TRANSACTION;
  END TRY BEGIN CATCH
    ROLLBACK TRANSACTION;    -- undoes BOTH inserts
    PRINT 'FK violation → ROLLBACK – Atomicity confirmed.';
  END CATCH;
```

6.3 Consistency Demo

```
-- CHECK constraint: end_time must be > start_time
BEGIN TRANSACTION;
  BEGIN TRY
    INSERT INTO BOOKING (start_time, end_time, ...)
    VALUES ('2025-06-02 11:00', '2025-06-02 09:00', ...); -- end BEFORE start!
    COMMIT TRANSACTION;
  END TRY BEGIN CATCH
    ROLLBACK TRANSACTION;
    PRINT 'CHECK constraint fired. DB stays consistent.';
  END CATCH;
```

6.4 Isolation Levels

Level	Meaning	Our Usage
READ COMMITTED	Only see committed data (no dirty reads)	Default — used for all dashboard SELECT queries
SERIALIZABLE	Full lock; no phantom reads	Used for counting pending bookings — ensures no new rows appear mid-count

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
BEGIN TRANSACTION;
  SELECT COUNT(*) AS pending FROM BOOKING WHERE status = 'Pending';
```

```
COMMIT TRANSACTION;
```

6.5 Deadlock Demo (run in 2 SSMS windows simultaneously)

```
-- Session A (Window 1):
BEGIN TRANSACTION;
    UPDATE BOOKING SET status='Approved' WHERE booking_id=4; -- locks row 4
    WAITFOR DELAY '00:00:05';
    UPDATE BOOKING SET status='Cancelled' WHERE booking_id=5; -- wants row 5 (B has it)
COMMIT TRANSACTION;

-- Session B (Window 2):
BEGIN TRANSACTION;
    UPDATE BOOKING SET status='Approved' WHERE booking_id=5; -- locks row 5
    WAITFOR DELAY '00:00:05';
    UPDATE BOOKING SET status='Cancelled' WHERE booking_id=4; -- wants row 4 (A has it)
COMMIT TRANSACTION;
-- SQL Server detects circular wait → kills one transaction (deadlock victim)
-- Msg 1205: Transaction was deadlocked... chosen as the deadlock victim.
```

6.6 Two-Phase Locking (2PL)

Growing phase: acquire all locks, never release. **Shrinking phase:** release locks, never acquire new ones. COMMIT triggers the shrinking phase and releases ALL locks at once.

```
BEGIN TRANSACTION;
    -- Growing: acquire S-lock (shared read)
    SELECT * FROM BOOKING WHERE booking_id = 4;

    -- Growing: escalate to X-lock (exclusive write)
    UPDATE BOOKING SET status = 'Approved' WHERE booking_id = 4;

    -- Shrinking: COMMIT releases ALL locks simultaneously
    COMMIT TRANSACTION;

-- HOLDLOCK and UPDLOCK hints to control lock type explicitly:
SELECT * FROM BOOKING WITH (HOLDLOCK) WHERE booking_id = 4; -- S-lock held until COMMIT
SELECT * FROM BOOKING WITH (UPDLOCK) WHERE booking_id = 4; -- upgrade-intent lock
```

7. backend_api.py — FastAPI Backend Explained

Python REST API running on `http://127.0.0.1:8000`. The React frontend calls it. It handles authentication, all database queries, and CRUD operations.

7.1 Database Connection Setup

```
import urllib
from sqlalchemy import create_engine, text

SERVER = r'rakhal\RSQL' # raw string r'' keeps backslash literal
DATABASE = 'SmartCampusDB'

# urllib.parse.quote_plus URL-encodes the backslash in server name
# Without this, ODBC misinterprets the backslash in connection string
params = urllib.parse.quote_plus(
    f"DRIVER={{ODBC Driver 17 for SQL Server}};"
    f"SERVER={SERVER};DATABASE={DATABASE};Trusted_Connection=yes;"
)
engine = create_engine(f"mssql+pyodbc:///odbc_connect={params}")
```

engine.begin() = auto-commit transaction (use for INSERT/UPDATE/DELETE). **engine.connect()** = read-only (use for SELECT). SQLAlchemy handles connection pooling automatically.

7.2 JWT Authentication Flow

```
from jose import JWTError, jwt

SECRET_KEY = "smartcampus_iiit_dharwad_da264_secret_2025"
ALGORITHM = "HS256"

def create_token(username, role):
    expire = datetime.utcnow() + timedelta(minutes=480) # 8-hour token
    payload = {"sub": username, "role": role, "exp": expire}
    return jwt.encode(payload, SECRET_KEY, algorithm=ALGORITHM)
    # Returns: base64(header).base64(payload).HMAC_SHA256(header+payload, SECRET_KEY)

def verify_token(credentials = Depends(security)):
    try:
        payload = jwt.decode(credentials.credentials, SECRET_KEY, algorithms=[ALGORITHM])
        username = payload.get("sub")
        if username not in AUTHORIZED_USERS:
            raise HTTPException(status_code=401, detail="Invalid token")
        return AUTHORIZED_USERS[username] # returns user dict {role, can_insert, ...}
    except JWTError:
        raise HTTPException(status_code=401, detail="Token expired or invalid")
```

Depends(verify_token) = FastAPI dependency injection. Any route decorated with `user=Depends(verify_token)` auto-runs `verify_token` first. If token invalid, route never executes — returns 401 automatically.

7.3 Pydantic Validation Models

```
from pydantic import BaseModel, Field

class EnergyReadingIn(BaseModel):
    meter_id: int
    kwh_consumed: float = Field(gt=0, le=100) # must be 0-100 kwh
    voltage: float = Field(ge=180, le=260) # must be 180-260 V
    peak_flag: int = Field(ge=0, le=1) # only 0 or 1

class AccessLogIn(BaseModel):
    user_id: int
    room_id: int
    access_method: str = Field(pattern="^(RFID|Biometric|PIN|Manual)$")
```

```
# regex enforces only valid auth methods
```

FastAPI validates incoming JSON against these models automatically. Wrong types or out-of-range values return HTTP 422 Unprocessable Entity before any database code runs.

7.4 POST /api/login

```
@app.post("/api/login")
def login(req: LoginRequest):
    user = AUTHORIZED_USERS.get(req.username)
    if not user or user["password"] != req.password:
        raise HTTPException(status_code=401, detail="Wrong username or password")
    token = create_token(req.username, user["role"])
    return {
        "token": token, # JWT stored in browser localStorage
        "username": req.username,
        "full_name": user["full_name"],
        "role": user["role"], # "admin" or "viewer"
        "can_insert": user["can_insert"], # True for admins, False for viewers
    }
```

7.5 GET /api/dashboard-stats — Powers the Entire Dashboard

One endpoint that runs 15+ SQL queries and returns everything the dashboard needs as one JSON object. Key SQL used:

```
with engine.connect() as conn:
    # Scalar queries for KPI cards
    total_energy = conn.execute(text("SELECT SUM(kwh_consumed) FROM fact_energy_consumption")).scalar()

    booking_count = conn.execute(text("SELECT COUNT(*) FROM BOOKING")).scalar()
    ticket_count = conn.execute(text(
        "SELECT COUNT(*) FROM MAINTENANCE_TICKET WHERE status IN ('Open','In Progress')
    )).scalar()

    # Anomaly detection with LAG window function
    anomaly = pd.read_sql("""
        SELECT TOP 20 meter_id, timestamp, kwh_consumed,
            kwh_consumed - LAG(kwh_consumed)
                OVER (PARTITION BY meter_id ORDER BY timestamp) AS delta_kwh,
            CASE WHEN kwh_consumed - LAG(kwh_consumed)
                OVER (PARTITION BY meter_id ORDER BY timestamp) > 2
            THEN 'SPIKE DETECTED' ELSE 'Normal' END AS anomaly_flag
        FROM ENERGY_READING ORDER BY timestamp DESC
    """, conn).to_dict(orient='records') # pandas → list of dicts → JSON-serialisable
```

7.6 POST /api/booking — With Conflict Detection

```
@app.post("/api/booking")
def create_booking(b: BookingIn, user=Depends(verify_token)):
    if not user["can_insert"]:
        raise HTTPException(status_code=403, detail="No insert permissions")
    with engine.begin() as conn: # auto-commit transaction
        # Check for overlapping bookings first
        conflict = conn.execute(text("""
            SELECT COUNT(*) FROM BOOKING
            WHERE room_id = :room
            AND status NOT IN ('Cancelled','Rejected')
            AND start_time < :end AND end_time > :start
        """), {"room": b.room_id, "start": b.start_time, "end": b.end_time}).scalar()
        if conflict:
            raise HTTPException(status_code=409, detail="Room already booked")
        # No conflict – insert
        conn.execute(text("""
            INSERT INTO BOOKING (user_id, room_id, start_time, end_time, purpose, status, created_at)
        """))
```

```
VALUES (:user, :room, :start, :end, :purpose, :status, GETDATE())
"""), {...}) # trg_booking_insert trigger fires automatically here
return {"success": True, "message": "Booking created"}
```

7.7 POST /api/run-etl — ETL Trigger Endpoint

```
@app.post("/api/run-etl")
def run_etl(user=Depends(verify_token)):
    if user["role"] != "admin": # role-based access control
        raise HTTPException(status_code=403, detail="Only admins can trigger ETL")
    import subprocess, sys
    result = subprocess.run(
        [sys.executable, "etl_pipeline.py"], # runs the Python script as subprocess
        capture_output=True, text=True, timeout=120
    )
    return {"success": result.returncode == 0, "log": result.stdout or result.stderr}
```

7.8 CORS Middleware

```
app.add_middleware(CORSMiddleware,
    allow_origins=["*"], # React (port 5173) can call API (port 8000)
    allow_methods=["*"], # GET, POST, PATCH, DELETE
    allow_headers=["*"], # Authorization header with JWT
)
# Without CORS, browsers block cross-origin requests (5173 → 8000) by default
```

8. Frontend — Login.jsx, App.jsx, LiveOps.jsx

8.1 Login.jsx

Standalone login page shown when no JWT exists in localStorage. Dark glassmorphism UI.

```
const [username, setUsername] = useState('');
const [password, setPassword] = useState('');
const [loading, setLoading] = useState(false);
const [error, setError] = useState('');

async function handleLogin(e) {
  e.preventDefault(); // prevent page reload on form submit
  setLoading(true);
  try {
    const res = await fetch('http://127.0.0.1:8000/api/login', {
      method: 'POST',
      headers: {'Content-Type': 'application/json'},
      body: JSON.stringify({ username, password }),
    });
    const data = await res.json();
    if (!res.ok) { setError(data.detail); return; }
    localStorage.setItem('sc_token', data.token); // store JWT
    localStorage.setItem('sc_user', JSON.stringify(data));
    onLogin(data); // callback prop from App.jsx → switches to dashboard
  } catch {
    setError('Cannot reach server — make sure backend is running on port 8000');
  } finally { setLoading(false); }
}
```

8.2 App.jsx — Core Architecture

~3000 lines. Manages all state, data fetching, navigation, and renders all 9 pages.

```
// Check if user is logged in (from localStorage)
const [user, setUser] = useState(() => {
  try { return JSON.parse(localStorage.getItem('sc_user')); }
  catch { return null; }
});

// Not logged in? Show login page
if (!user) return <LoginPage onLogin={u => setUser(u)} />;

// Fetch all dashboard data (with JWT auth header)
const fetchData = useCallback(async () => {
  try {
    const res = await fetch(API_URL, {
      headers: { 'Authorization': `Bearer ${localStorage.getItem('sc_token')}` }
    });
    if (res.status === 401) { setUser(null); return; } // token expired → logout
    setData(await res.json());
    setDemo(false);
  } catch {
    setData(MOCK); // API down → use hardcoded demo data
    setDemo(true); // show "Demo Mode" badge
  }
}, []);

// Auto-refresh every 30 seconds
useEffect(() => {
  fetchData();
  const interval = setInterval(fetchData, 30000);
  return () => clearInterval(interval); // cleanup on unmount
}, [fetchData]);
```

Important patterns: **useCallback** memoises `fetchData` so `setInterval` does not create a new function reference every render. **useMemo** memoises computed values like filtered booking lists. **Conditional rendering** `{page==='overview' && }` shows/hides pages without unmounting components unnecessarily.

8.3 MOCK Data — Demo Mode

If API is unreachable (catch block), `setData(MOCK)` is called. **MOCK** is a large hardcoded object at the top of `App.jsx` with realistic values matching all chart shapes. This is how the screenshots look — they are from Demo Mode with **MOCK** data.

```
const MOCK = {
  kpis: { energy: 87.34, rooms: 16, users: 109, bookings: 200, tickets: 18, ... },
  anomaly_readings: [
    { meter_id: 4, kwh_consumed: 9.20, delta_kwh: 3.6, anomaly_flag: 'SPIKE DETECTED' },
    { meter_id: 9, kwh_consumed: 9.48, delta_kwh: 4.2, anomaly_flag: 'SPIKE DETECTED' },
  ],
  buildings: [
    { building_name: 'E Block', total: 4820, peak_kwh: 2410 },
    { building_name: 'PI Block', total: 1230, peak_kwh: 610 }, ...
  ],
};
```

8.4 Recharts Example — Building Energy Bar Chart

```
import { BarChart, Bar, XAxis, YAxis, Tooltip, ResponsiveContainer } from 'recharts';

// data.buildings = [{building_name: 'E Block', total: 4820}, ...]
<ResponsiveContainer width="100%" height={300}>
  <BarChart data={data.buildings}>
    <XAxis dataKey="building_name" />
    <YAxis />
    <Tooltip />
    <Bar dataKey="total" fill="#3B82F6" radius={[4,4,0,0]} />
  </BarChart>
</ResponsiveContainer>
```

8.5 LiveOps.jsx — All Four Forms + ETL Trigger

loadingMap tracks which forms are submitting independently — "Log Energy" spinner does not affect "Create Booking" button.

```
// Generic POST helper used by ALL four forms
async function post(formKey, endpoint, payload, successMsg, resetFn) {
  setLoadingMap(p => ({...p, [formKey]: true}));
  try {
    const res = await fetch(`${API}/${endpoint}`, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        'Authorization': `Bearer ${localStorage.getItem('sc_token')}`
      },
      body: JSON.stringify(payload),
    });
    const data = await res.json();
    if (!res.ok) throw new Error(data.detail);
    setToast({ msg: successMsg, type: 'success' }); // green toast notification
    resetFn(); // clear form fields
    onRefresh(); // tell App.jsx to re-fetch dashboard data
  } catch (err) {
    setToast({ msg: err.message, type: 'error' }); // red error toast
  } finally {
    setLoadingMap(p => ({...p, [formKey]: false}));
  }
}
```

```
// Example: Log Energy Reading button onClick
onClick={() => post('energy', 'energy-reading', {
  meter_id:      parseInt(energyForm.meter_id),
  kwh_consumed:  parseFloat(energyForm.kwh),
  voltage:       parseFloat(energyForm.voltage),
  peak_flag:     energyForm.peak === 'Yes' ? 1 : 0,
}, 'Energy reading inserted!', () => setEnergyForm(defaultEnergyForm))}
```

Live Feed polling: Every 10 seconds `useEffect` calls `GET /api/live-feed`. Returns 15 most recent events (energy, bookings, tickets, access logs) sorted by timestamp. Displayed in the scrollable activity feed.

```
// 30-second auto-refresh countdown
function RefreshTimer({ interval=30, onTick }) {
  const [secs, setSecs] = useState(interval);
  useEffect(() => {
    const iv = setInterval(() => {
      setSecs(s => {
        if (s <= 1) { onTick(); return interval; } // trigger full refresh
        return s - 1;
      });
    }, 1000);
    return () => clearInterval(iv); // cleanup prevents memory leak
  }, [interval, onTick]);
  return <span>{secs}s</span>;
}
```

9. ETL.sql — SQL-Based Data Warehouse Loading

ETL.sql is the pure SQL version of the ETL process. It runs in SSMS to populate the warehouse tables from the OLTP tables. (etl_pipeline.py does the same thing automatically from Python.)

9.1 The Star Schema

Table	Type	Purpose	Rows
fact_energy_consumption	FACT	One row per energy reading — stores kwh, voltage, peak_flag, date_key, room_key, meter_key	515+
dim_date	DIMENSION	Every day in 2025 with day/month/quarter/year attributes	365
dim_room	DIMENSION	Denormalised: room + floor + building + dept all in one flat row	16
dim_meter	DIMENSION	Meter type + associated room	27
dim_building	SNOWFLAKE	Sub-dimension of dim_room (Snowflake normalisation)	6
dim_department	SNOWFLAKE	Sub-dimension of dim_building	3

9.2 dim_date Population

```
INSERT INTO dim_date
SELECT
    CAST(FORMAT(d, 'yyyyMMdd') AS INT) AS date_key, -- 20250315 = 15 Mar 2025
    d AS full_date,
    DATENAME(WEEKDAY, d) AS day_name,
    DATEPART(QUARTER, d) AS quarter,
    YEAR(d) AS year,
    CASE WHEN DATEPART(WEEKDAY,d) IN (1,7) THEN 1 ELSE 0 END AS is_weekend
FROM (
    SELECT DATEADD(DAY, n, '2025-01-01') AS d -- add 0..364 days to Jan 1
    FROM (
        SELECT TOP 365 ROW_NUMBER() OVER (ORDER BY (SELECT NULL))-1 AS n
        FROM sys.objects
    ) nums
) dates;
```

✓ **Tip:** date_key is an integer (20250315) not a DATE. Integer JOINS are faster than DATE JOINS in high-volume warehouse queries.

9.3 dim_room — Intentional Denormalisation

```
INSERT INTO dim_room (room_id, room_no, room_type, capacity,
    floor_label, building_name, building_type, dept_name)
SELECT R.room_id, R.room_no, R.room_type, R.capacity,
    F.floor_label,
    B.bld_name, B.bld_type,
    D.dept_name
FROM ROOM R
JOIN FLOOR F ON R.floor_id = F.floor_id
JOIN BUILDING B ON F.building_id = B.building_id
JOIN DEPARTMENT D ON B.dept_id = D.dept_id;
-- 4-table JOIN collapsed into 1 flat warehouse row
-- OLAP queries need only JOIN fact to dim_room — no chaining
```

9.4 Fact Table Load

```
INSERT INTO fact_energy_consumption
(date_key, room_key, meter_key, kwh_consumed, voltage, peak_flag)
SELECT
    CAST(FORMAT(CAST(E.timestamp AS DATE), 'yyyyMMdd') AS INT) AS date_key,
```

```

    DR.room_key,    -- warehouse surrogate key (not OLTP room_id)
    DM.meter_key,   -- warehouse surrogate key (not OLTP meter_id)
    E.kwh_consumed, E.voltage, E.peak_flag
FROM ENERGY_READING E
JOIN ENERGY_METER M ON E.meter_id = M.meter_id
JOIN dim_meter      DM ON DM.meter_id = M.meter_id
JOIN dim_room       DR ON DR.room_id = M.room_id;
-- cost_inr is a computed column: AS (kwh_consumed * 8.5) -- no ETL step needed

```

9.5 OLAP Operations

ROLL-UP — GROUP BY GROUPING SETS for multi-level aggregation:

```

SELECT DD.year, DD.quarter, DD.month_name, SUM(F.kwh_consumed) AS total_kwh
FROM fact_energy_consumption F JOIN dim_date DD ON F.date_key = DD.date_key
GROUP BY GROUPING SETS (
    (DD.year, DD.quarter, DD.month_name), -- month subtotal
    (DD.year, DD.quarter),                -- quarter subtotal
    (DD.year),                            -- year subtotal
    ()                                     -- grand total
);

```

DICE — filter on 3 dimensions simultaneously:

```

SELECT DR.room_no, SUM(F.kwh_consumed) AS kwh
FROM fact_energy_consumption F
JOIN dim_date  DD ON F.date_key = DD.date_key
JOIN dim_room  DR ON F.room_key = DR.room_key
JOIN dim_meter DM ON F.meter_key = DM.meter_key
WHERE DD.year=2025 AND DD.quarter=1      -- time filter
      AND DR.room_type = 'Computer Lab'  -- room type filter
      AND DM.meter_type = 'Electricity'; -- meter type filter

```

10. etl_pipeline.py — Automated Python ETL

Python script that runs automatically via POST /api/run-etl. Uses pandas for data manipulation and SQLAlchemy for DB connectivity. Implements incremental (watermark-based) loading with IQR outlier detection.

10.1 Watermark-Based Incremental Load

```
def get_watermark(self):
    """Returns date of last row already in warehouse."""
    with self.engine.connect() as conn:
        result = conn.execute(text("""
            SELECT MAX(full_date) FROM dim_date
            WHERE date_key IN (SELECT DISTINCT date_key FROM fact_energy_consumption)
            """)).scalar()
        if result: return result.strftime('%Y-%m-%d %H:%M:%S')
        return '2000-01-01' # Empty warehouse: load everything

def extract_readings(self, watermark):
    """Pull only NEW readings (after last watermark)."""
    return pd.read_sql(f"""
        SELECT reading_id, timestamp, kwh_consumed, voltage, peak_flag, meter_id
        FROM ENERGY_READING
        WHERE timestamp > '{watermark}' -- only rows added since last ETL run
        """, self.engine)
```

10.2 IQR Outlier Detection

```
def clean_data(self, df):
    initial = len(df)
    df['kwh_consumed'] = df['kwh_consumed'].fillna(df['kwh_consumed'].mean()) # fill NULLs
    Q1 = df['kwh_consumed'].quantile(0.25)
    Q3 = df['kwh_consumed'].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR # fence: anything below is outlier
    upper = Q3 + 1.5 * IQR # fence: anything above is outlier
    df = df[(df['kwh_consumed'] >= lower) & (df['kwh_consumed'] <= upper)]
    logger.info(f"IQR: removed {initial - len(df)} outliers.")
    return df
# IQR = Q3 - Q1 = 75th percentile - 25th percentile
# Any reading more than 1.5 × IQR beyond the quartiles is discarded
```

✓ **Tip:** "Why 1.5×IQR?" This is the standard Tukey fences formula. Values beyond this range are statistically unlikely to be real measurements — they are almost certainly sensor errors or transmission glitches.

10.3 Transform — Surrogate Key Mapping

```
def transform(self, df):
    dim_room = pd.read_sql("SELECT room_id, room_key FROM dim_room", self.engine)
    dim_meter = pd.read_sql("SELECT meter_id, meter_key FROM dim_meter", self.engine)
    meter_room = pd.read_sql("SELECT meter_id, room_id FROM ENERGY_METER", self.engine)

    # Join to get room_id from meter_id
    df = df.merge(meter_room, on='meter_id')
    # Map OLTP room_id → warehouse room_key
    df = df.merge(dim_room, on='room_id') # adds room_key column
    # Map OLTP meter_id → warehouse meter_key
    df = df.merge(dim_meter, on='meter_id') # adds meter_key column
    # Generate date_key as YYYYMMDD integer
    df['timestamp'] = pd.to_datetime(df['timestamp'])
    df['date_key'] = df['timestamp'].dt.strftime('%Y%m%d').astype(int)
    return df[['date_key', 'room_key', 'meter_key', 'kwh_consumed', 'voltage', 'peak_flag']]
```

10.4 Load — Bulk Insert to Fact Table

```
def load(self, df):
    if df.empty: return
    # if_exists='append' → add rows (don't overwrite existing)
    # index=False → don't write pandas row index (fact_id is IDENTITY auto-assign)
    df.to_sql('fact_energy_consumption', self.engine, if_exists='append', index=False)

def run(self):
    watermark = self.get_watermark()
    raw_data = self.extract_readings(watermark)
    if not raw_data.empty:
        cleaned = self.clean_data(raw_data)
        transformed = self.transform(cleaned)
        self.load(transformed)
    else:
        logger.info("Warehouse already up to date.")
```

11. Likely Questions & Model Answers

Q1: Why BCNF and not just 3NF?

3NF still allows FDs where the determinant is not a superkey. BCNF eliminates all such cases — every determinant must be a candidate key. All our 12 OLTP tables satisfy BCNF because every non-key attribute is fully determined only by the PK.

Q2: What is a surrogate key and why use it in the warehouse?

A surrogate key (room_key, meter_key) is a warehouse-internal integer ID independent of the OLTP PK. It decouples the warehouse from operational changes — if OLTP room_ids are renumbered, warehouse history stays intact with its own stable keys.

Q3: How does the ETL pipeline avoid loading duplicates?

Watermark-based incremental load: `SELECT MAX(full_date) from dim_date` joined with `fact_energy_consumption` gives the last loaded date. Only `ENERGY_READING` rows with `timestamp > watermark` are extracted. No duplicate loading.

Q4: How do you prevent double-booking?

`usp_AddBooking` checks: `IF EXISTS (SELECT 1 FROM BOOKING WHERE room_id=@room AND status NOT IN (Cancelled,Rejected) AND start_time < @end AND end_time > @start) → RAISERROR + ROLLBACK`. The entire check+insert is wrapped in `BEGIN TRANSACTION` with appropriate locking.

Q5: Why is MAINTENANCE_TICKET a weak entity?

A ticket is meaningless without the room it belongs to — `room_id` is NOT NULL and part of its logical identity. Deleting the room would orphan all its tickets. This is the identifying relationship / weak entity pattern from ER theory.

Q6: Difference between trigger and stored procedure?

Stored procedure is called explicitly (`EXEC usp_AddBooking`). Trigger fires automatically on a DML event (`INSERT/UPDATE/DELETE`) on a specific table — invisible to the calling application. Our triggers auto-populate `AUDIT_TRAIL` without any application code knowing.

Q7: How does JWT work?

Login → server creates `JWT = base64(header) + "." + base64(payload) + "." + HMAC_SHA256(header+payload, SECRET_KEY)`. Frontend stores token, sends it in Authorization header on every request. Server verifies HMAC signature — no DB lookup needed on each request.

Q8: Why use React instead of plain HTML?

React's virtual DOM efficiently re-renders only changed parts. With 15+ charts refreshing every 30 seconds, plain HTML would re-render everything on each update. React also enables component reuse (`FormCard`, `Field`, `Btn` reused in all four Live Operations forms).

Q9: SLICE vs DICE in OLAP?

SLICE: fix ONE dimension (`WHERE year=2025`) and show all rooms. **DICE:** fix MULTIPLE dimensions simultaneously (`WHERE year=2025 AND quarter=1 AND room_type=Computer Lab AND meter_type=Electricity`). **SLICE** is one filter; **DICE** is multiple filters creating a sub-cube.

Q10: Why `engine.begin()` for writes and `engine.connect()` for reads?

`engine.begin()` wraps in a transaction that auto-commits on success and auto-rolls-back on exception. `engine.connect()` is a read connection with no transaction overhead. Using `begin()` for writes ensures partial inserts never persist if an error occurs mid-way.

Q11: What does the LAG() window function do?

LAG(column) OVER (PARTITION BY meter_id ORDER BY timestamp) returns the value of "column" from the PREVIOUS row within each meter group ordered by time. Subtracting current from previous gives the delta — if $\text{delta} > 2 \text{ kWh}$ we flag SPIKE DETECTED for anomaly detection.

Q12: What is CORS and why do you need it?

CORS = Cross-Origin Resource Sharing. Browsers block JavaScript from making requests to a different origin (React on localhost:5173 calling API on localhost:8000). CORSMiddleware in FastAPI adds response headers (Access-Control-Allow-Origin: *) telling the browser to allow the cross-origin request.

12. Quick Reference — Files, Commands, Endpoints

12.1 Files in Order

File	What It Is	When to Run
DBMS.sql	Creates DB, tables, procedures, triggers, warehouse, OLAP	First — once to set up everything
seed_bulk.sql	Inserts 500+ realistic rows for demo charts	Second — once
transaction.sql	ACID demos — run blocks in SSMS	For DA264 Unit III demonstration
ETL.sql	SQL-only ETL — loads warehouse manually	Optional (Python pipeline does same thing)
backend_api.py	FastAPI server on port 8000	python backend_api.py (before opening dashboard)
auth_users.py	User credentials dict (auto-imported by backend)	No separate run needed
etl_pipeline.py	Python ETL script	Auto-called by /api/run-etl endpoint
main.jsx	React entry point	npm run dev
App.jsx	Main dashboard (9 pages)	Auto-loaded by main.jsx
LiveOps.jsx	Live Operations page	Auto-imported into App.jsx
login.jsx	Login page	Auto-imported into App.jsx

12.2 Startup Commands

```
# 1. Open SSMS → run DBMS.sql → run seed_bulk.sql

# 2. Start the backend API:
python backend_api.py
# → Listening on http://127.0.0.1:8000
# → Swagger docs at http://127.0.0.1:8000/docs

# 3. Start the React frontend:
npm run dev
# → Dashboard at http://localhost:5173

# Credentials:
# rakhal / rakhal@admin123 → Admin (can insert data)
# admin1 / admin1@smart456 → Admin
# rishikesh / admin2@smart789 → Viewer (read-only, no insert)
```

12.3 API Endpoints

Method	Endpoint	Auth Needed	Action
POST	/api/login	None	Get JWT token
GET	/api/dashboard-stats	JWT	All dashboard data (15+ queries)
GET	/api/live-feed	JWT	Last 15 events for activity feed
POST	/api/energy-reading	JWT + can_insert	Insert ENERGY_READING row
POST	/api/booking	JWT + can_insert	Create BOOKING (with conflict check)
PATCH	/api/booking/status	JWT + can_insert	Update booking status
POST	/api/maintenance-ticket	JWT + can_insert	Raise MAINTENANCE_TICKET

Method	Endpoint	Auth Needed	Action
PATCH	/api/maintenance-ticket/status	JWT + can_insert	Update ticket status
POST	/api/access-log	JWT + can_insert	Log ACCESS_LOG entry
POST	/api/run-etl	JWT + admin role	Trigger etl_pipeline.py